

VisionInspect AI — Project & Milestone-3 Status Report

Om Shakthi Vemuganti
(Team-2) — Infosys Springboard Internship

1. Project Overview

VisionInspect AI is an AI-powered manufacturing quality inspection platform built as part of the Infosys Springboard internship. A Quality Engineer or Factory Supervisor uploads product images through the platform; the system runs anomaly detection to compare each image against a defect-free reference, localizes and classifies any defect present, scores its severity, and returns an automated pass, rework, or reject decision. The platform also includes a dashboard for inspection history and production-quality analytics, giving teams visibility into defect trends over time.

The system is built around anomaly detection using the PatchCore model on the MVTec AD dataset, covering 15 product categories such as bottles, zippers, tiles, and transistors. Detected defects are scored using a weighted severity formula (Size 30%, Location 25%, Defect Type 25%, Confidence 20%), which maps to four action tiers — Critical, High, Medium, and Low — driving the pass/rework/reject decision shown to the user.

Architecturally, the backend follows a polyglot persistence design: PostgreSQL stores structured, relational data — users, inspection summaries, final severity scores, pass/fail decisions, and reports — while MongoDB stores flexible, nested AI model output such as raw anomaly scores, heatmap and mask file paths, and variable-length lists of localized defect detections. The two databases are linked by a shared `inspection_id`, allowing the relational and document layers to stay in sync without duplicating data.

2. Team Roles & Responsibilities

Team Member	Role & Responsibility
Yash Goyal	AI/ML Lead — dataset pipeline across all 15 categories, PatchCore anomaly-detection model, and lead integration/bug-fixing across the full stack.
Arnab Ghosal	Backend & Database Lead — system architecture, PostgreSQL schema design, category dataset preprocessing.
Himabindhu Ravuri	Frontend & API Integration — React dashboard, login, upload and history pages; co-led final integration and bug-fixing.
Lakshyatha Chamarthi	Backend & Auth — Flask backend, REST API endpoints, JWT authentication flow.
Om Shakthi Vemuganti (me)	Coordination & MongoDB Integration — task-division facilitation, dataset sourcing, MongoDB module (Atlas setup, connection layer, schema linked via <code>inspection_id</code>), consolidated documentation.

3. Team Work Completed Across the Project

The table below summarizes what each team member has delivered across the project so far, from initial architecture and dataset work through to the current, near fully-integrated system.

Team Member	Work Completed	Status
Yash Goyal	Dataset pipeline & preprocessing (all 15 categories), PatchCore model proof-of-concept, and lead role in Milestone-3 integration and bug-fixing across backend, frontend, and databases.	Done
Arnab Ghosal	System architecture, PostgreSQL 5-table schema, severity-scoring logic, category dataset preprocessing.	Done
Lakshyatha Chamarthya	Flask backend with 12 REST endpoints, JWT authentication, automated Pytest suite (10/10 passing).	Done
Himabindhu Ravuri	React frontend (login, upload, dashboard, history), co-led Milestone-3 integration and continuous bug-fixing.	Done
Om Shakthi Vemuganti (me)	Task-division coordination, dataset sourcing, full MongoDB integration module; explored UI improvements this milestone (not adopted — see Section 5).	See note

4. Milestone-Wise Progress

Milestone 1 — Architecture, Dataset & Foundations

The team finalized the project architecture, set up the layered folder structure, and began work on the PostgreSQL schema. The full MVTec AD dataset (15 categories, ~4.92 GB) was sourced and shared across the team. Dataset categories were divided among members for individual preprocessing and exploration, and early groundwork was laid for the Flask backend and React frontend.

Milestone 2 — Core Modules Built

Yash completed dataset preprocessing and exploration across all 15 categories and trained the PatchCore anomaly-detection model as a proof of concept on the Bottle category, achieving clear separation between normal and defective predictions. Arnab designed and implemented the 5-table PostgreSQL schema along with the severity-scoring logic as executable code. Lakshyatha built out the Flask backend with 12 REST endpoints and JWT authentication, backed by a full automated test suite. Himabindhu delivered the React frontend — login, upload, dashboard, and history pages — with all backend calls isolated for easy integration later. In parallel, MongoDB integration was completed end-to-end: Atlas cluster setup, the pymongo connection layer, and a document schema for raw AI output, linked to PostgreSQL via a shared inspection_id.

Milestone 3 — Integration & Stabilization

This milestone centered on integrating the previously independent modules into a single working pipeline. Yash and Himabindhu led an intensive, multi-day effort to connect the frontend, backend, and databases, resolving issues such as the report API/database mismatch and the SQLite-vs-PostgreSQL conflict left over from the initial backend scaffold. By the end of the milestone, the system was reported as fully integrated, cleaned up, and demo-ready, with deployment targeted for the following week.

5. My Contribution This Milestone

With my originally assigned dataset categories already absorbed into the team's full-pipeline run, I explored an additional area for this milestone: improving the frontend UI using Claude Code, aiming to enhance the visual design and usability of the existing inspection dashboard.

However, the proposed UI changes did not fit into the project, as the existing UI was already tightly integrated with the backend, and restructuring it risked breaking working connections this close to the milestone deadline.

6. Next Steps for Milestone 4

- Complete deployment of the integrated system (backend, frontend, PostgreSQL, and MongoDB) to a cloud environment.
- Finalize and consolidate project documentation, including setup instructions, architecture overview, and API/database reference.
- Review and harden the MongoDB integration for production use, including connection security and environment configuration.
- Conduct end-to-end testing of the fully integrated pipeline ahead of the final demo.
- Prepare the final milestone presentation and demo script covering all modules.